

Spec 02 — Motor de reserva (spec central)

O coração do BdayList. Reserva **atômica, idempotente, anti-TOCTOU**, com auditoria e suporte a presente em grupo.

1. Garantias

Propriedade	Mecanismo
Atomicidade	Transação Postgres com <code>SELECT ... FOR UPDATE</code> no presente
Anti-TOCTOU	Check de disponibilidade e INSERT na mesma transação
Idempotência	<code>UNIQUE (idempotency_key)</code> + replay explícito antes da transação
Sem duplicata (comum)	Índice único parcial em reservas ativas
Sem duplicata (grupo)	<code>SUM(contribution)</code> recalculado na transação antes de mudar status
Auditoria	Linha em <code>reservation_log</code> na mesma transação

```
-- presente comum: no máx. 1 reserva ATIVA por slot
create unique index reservations_single_gift_unique
  on reservations (gift_id, slot) where status = 'active';
-- mesmo convidado não duplica
create unique index reservations_one_per_guest
  on reservations (gift_id, guest_id) where status = 'active';
```

2. `reserve_gift` — pré/pós-condições e erros

Pré: `giftId` existe; presente `available` ; `idempotencyKey` válido; se grupo, `contribution > 0` . **Pós (sucesso):** reserva `active` inserida; log `created` ; status atualizado (comum → `reserved` ; grupo → `reserved` se meta atingida).

Código	HTTP	Condição
<code>GIFT_NOT_FOUND</code>	404	id inexistente
<code>ALREADY_RESERVED</code>	409	comum já reservado
<code>GROUP_FULL</code>	409	grupo já atingiu a meta
<code>VALIDATION_ERROR</code>	422	campos inválidos / <code>contribution</code> ausente em grupo
<code>IDEMPOTENT_REPLAY</code>	200	mesmo <code>idempotencyKey</code> → retorna a reserva original

3. Função (Postgres)

```

create or replace function reserve_gift(
  p_gift_id uuid, p_guest_name text, p_guest_email text,
  p_message text, p_idempotency_key text, p_contribution numeric default null
) returns json language plpgsql security definer as $$
declare v_gift gifts%rowtype; v_res reservations%rowtype; v_collected numeric;
begin
  -- 1. replay idempotente (caminho mais barato, antes do lock)
  select * into v_res from reservations where idempotency_key = p_idempotency_key;
  if found then return row_to_json(v_res); end if;

  -- 2. lock do presente para serializar concorrentes
  select * into v_gift from gifts where id = p_gift_id for update;
  if not found then raise exception 'GIFT_NOT_FOUND' using errcode='P0001'; end if;

  if v_gift.status = 'reserved' then
    if not v_gift.is_group then raise exception 'ALREADY_RESERVED' using errcode='P0002';
    else raise exception 'GROUP_FULL' using errcode='P0003'; end if;
  end if;
  if v_gift.is_group and p_contribution is null then
    raise exception 'VALIDATION_ERROR' using errcode='P0004';
  end if;

  -- 3. criar reserva
  insert into reservations (gift_id, event_id, guest_name, guest_email, message, idempotency_key, contribution)
  values (p_gift_id, v_gift.event_id, p_guest_name, p_guest_email, p_message, p_idempotency_key, p_contribution)
  returning * into v_res;

  -- 4. auditoria (mesma transação)
  insert into reservation_log (reservation_id, action, actor_name)
  values (v_res.id, 'created', p_guest_name);

  -- 5. atualizar status
  if not v_gift.is_group then
    update gifts set status = 'reserved' where id = p_gift_id;
  else
    select coalesce(sum(contribution),0) into v_collected
    from reservations where gift_id = p_gift_id and status = 'active';
    if v_collected >= v_gift.group_target then
      update gifts set status = 'reserved' where id = p_gift_id;
    end if;
  end if;

  return row_to_json(v_res);
exception
  when unique_violation then
    raise exception 'ALREADY_RESERVED' using errcode='P0002';
end $$;

```

Por que **FOR UPDATE** E o índice único? O índice é a rede final; o **FOR UPDATE** faz a 2ª transação bloquear até a 1ª commitar, devolvendo **ALREADY_RESERVED** rastreável em vez de um erro de constraint genérico.

4. **cancel_reservation**

Lock da reserva ativa → valida identidade (no MVP: **guest_name** ; depois: **auth.uid()**) → **status='cancelled'**, **cancelled_at=now()** → log **cancelled** → reverte status do presente (comum → **available** ; grupo → **available** se **SUM < target**). Erros: **RESERVATION_NOT_FOUND** (404), **NOT_AUTHORIZED** (403).

5. BaaS vs backend dedicado

O SQL é idêntico. No Supabase roda como RPC (**supabase.rpc('reserve_gift', ...)**) via Edge Function com **service_role** . No backend dedicado, a mesma transação numa camada de repositório (**BEGIN ... COMMIT**). A diferença é só **onde o caller vive**.

6. Decisões abertas

- **Auth de convidado no cancelamento:** MVP usa `guest_name` (fraco mas aceitável). Migrar para `guest_user_id / auth.uid()` quando houver sessão de convidado.
- `groupGoal.collected` : calcular em runtime (`SUM`) no MVP; denormalizar só sob lentidão medida.
- **Overcommit em grupo:** aceitar e tratar na UI ("meta superada"), ou `CHECK` com tolerância.